

parsed a real document. We have 5 synthetic loans sitting as raw PDFs/XML that were specifically built with known defects — that's wasted unless something turns them into the engine's actual input shape, accurately.

WHAT EXISTS TODAY

- `p0/fixtures/golden.py` — 100% hand-authored `SourceValue` objects
- `p0/eval_synth/generator.py` — mutation-based, still hand-typed
- Zero code paths that read an actual PDF or MISMO file into the engine's shape
- 5 real synthetic loan packages sitting unused in `demo/syn/`

WHAT'S NEEDED

- Document-derived `CanonicalLoan` fixtures for all 5 loans
- Built once, offline ("dev mode") — never a runtime dependency
- Provably accurate — not just plausible-looking
- Reuses the engine's already-pinned input contract, invents nothing new

FIG 1

Extraction architecture — accepted path vs. rejected path

Olav's live `examples/mortgage-qc` pipeline was audited as a possible shortcut. Verdict: reuse the idea, not the code — it can't run standalone and its own issue log shows live accuracy problems. The accepted path is boring on purpose: deterministic text parsing, because these are born-digital synthetic PDFs with a clean text layer, not scans.

source / consumer accepted (deterministic) rejected / dead end

pinned contract (specs 001a/001b) hard accuracy gate

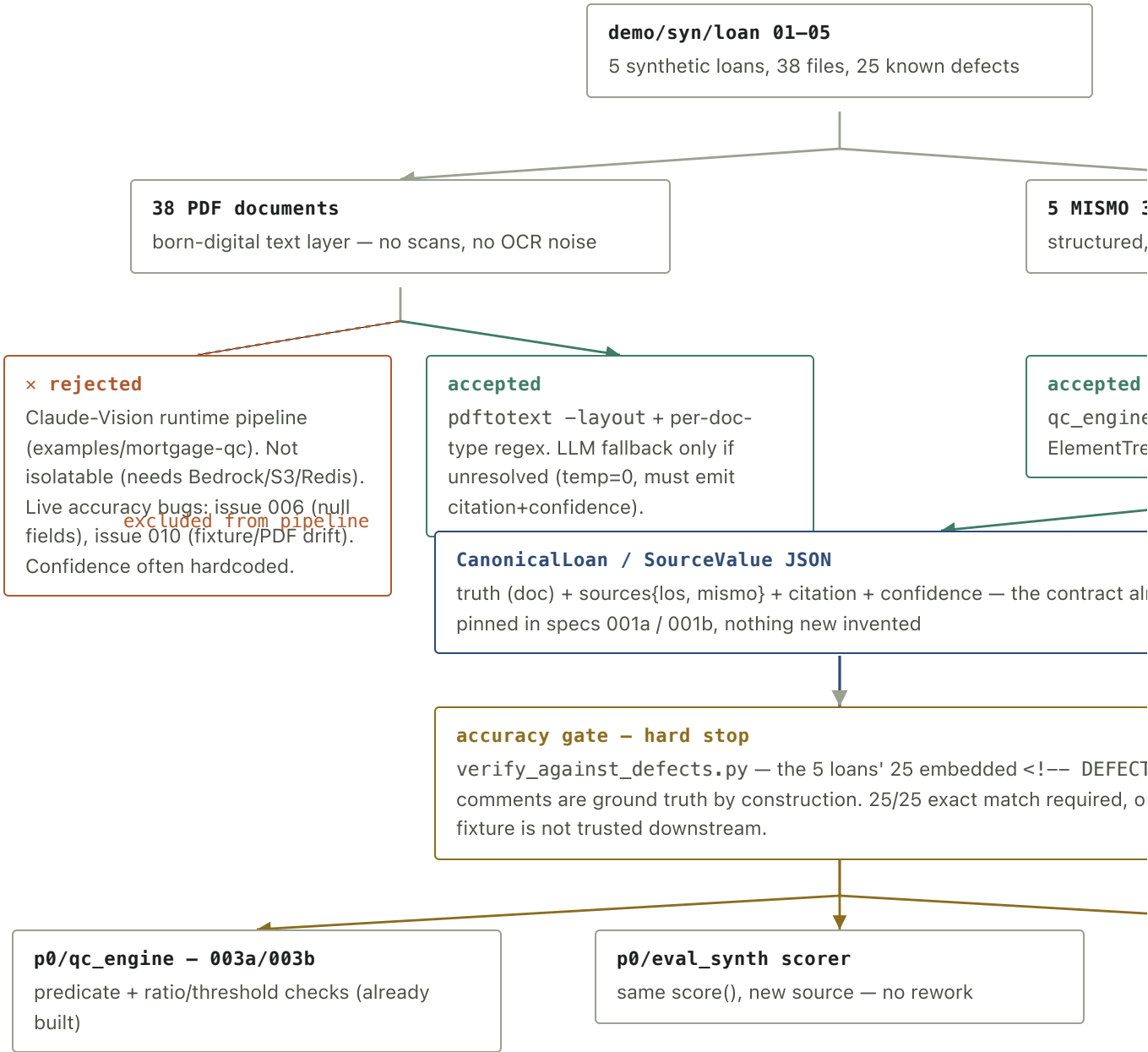


FIG 2

Field catalog — derived from real rules, not guessed from these 5 files

The 7-field seed catalog covers none of the 25 built-in defects. Rather than inventing new fields to fit these specific documents, each candidate field is checked against the actual rule taxonomy — `p0/eval_synth/taxonomy.json`, derived from the real 7,398-condition AMQ workbook — so the catalog grows to match what rules genuinely check for.



5 archetypes: UNDEFINED, THRESHOLD, MISMAATCH, EXPIRED, MISSING, INACCURATE, INCOMPLETE, POLICY

cross-reference

which archetype does each defect match? several are exact-text matches to a rule, e.g. "the manner in which title is held on the 1003 does not match the commitment"

~26 rule-grounded fields, cited

extends the 7-field seed catalog to ~33 — every field traceable to a real rule condition, none invented for convenience

doc-vs-doc gap flag

~5 fields are doc-vs-doc SourceValue model only h 003c, not silently forced.

SEC B

Where this stands right now

FIELD SCOPE

Extend the catalog

Rule-grounded via taxonomy.json, not guessed from the 5 loans. ~26 new fields, each cited to a real rule condition.

BUILD PROCESS

Lightweight script

Matches p0/eval_synth's existing convention — no formal spec cycle. The catalog edit itself still runs through 001a's own zero-regression gate.

OPEN, FLAGGED

Doc-vs-doc check kind

~5 fields need a comparison the engine doesn't have yet. Deferred to whoever specs 003c — not decided here.

SEC C

Build sequence

This part is a genuine sequence — each step gates the next.

1 Scope lock

Field catalog extension + build style, both now decided (above).

2 XML adapter

Extend `mismo.py` to emit every in-scope field for all 5 MISMO files.

gate: diff against hand-read values

3 PDF adapter

Per-document-type `pdftotext` + regex extractors across the ~10 document types.

gate: diff against hand-read values

4 Fixture assembly

Merge doc-side (truth) + MISMO-side (`sources.mismo`) into 5 CanonicalLoan JSON files.

gate: schema-valid against `field-catalog-schema.md`

5 Accuracy gate

All 25 DEFECT comments reproduce as expected mismatches, zero silent nulls.

gate: 25/25 exact – hard stop if not

6 Wire to downstream

Point `eval_synth/engine` tests at the new fixtures alongside the hand-authored golden set.

gate: zero regression on existing verdicts

What this proves, and what it doesn't: a 25/25 pass proves the extraction *mechanism* is accurate against known, constructed documents. It does not prove accuracy against real lenders' scanned/messy PDFs or real Touchless output — that remains gated on Kayla's real loans, same as everywhere else in this project. Full detail: `output/PLAN-synthetic-loan-extraction.md`.